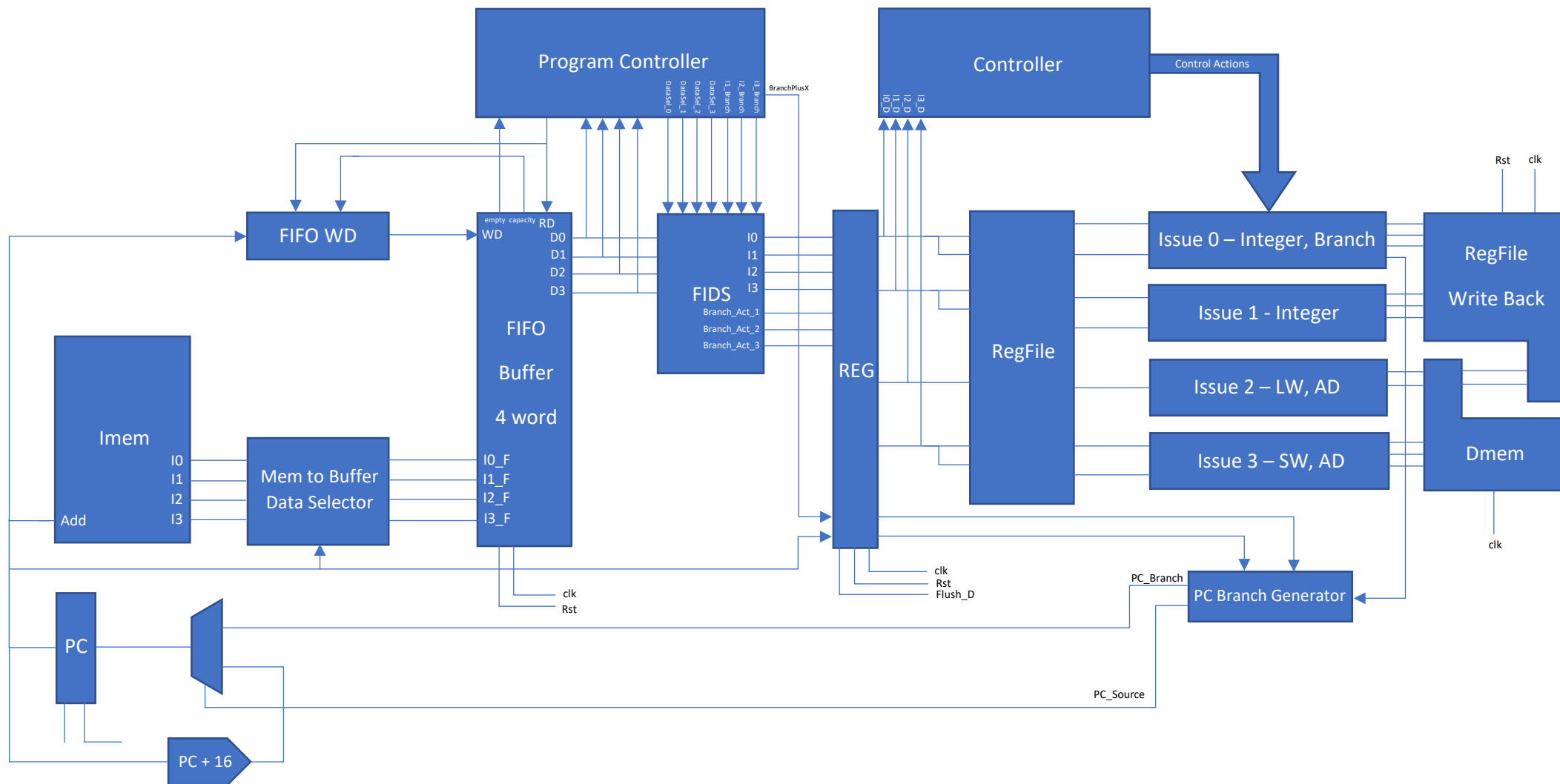


عنوان پروژه : طراحی پردازنده MIPS Four Issue

مشخصات طراحی:

- طراحی به صورت Four issue – Static scheduling – In order issue
- دارای Instruction memory با قابلیت خواندن بلاک های ۴ کلمه ای
- دارای رجیستر فایل با ۳ پورت نوشتن همزمان و ۷ پورت خواندن همزمان
- دارای Data memory بدون تاخیر
- طراحی به صورت 3 stage pipelined
- تخمین پرش به صورت Always not taken
- در هر پالس ساعت امکان دریافت و Fetch، ۴ دستور وجود دارد
- پیاده سازی دارای ۴ مسیر صدور و اجرای دستور به صورت زیر می باشد:
 - یک مسیر اجرای دستورات پرش و اعداد صحیح
 - یک مسیر اجرای دستورات اعداد صحیح
 - یک مسیر برای تولید آدرس و خواندن از حافظه
 - یک مسیر برای تولید آدرس و نوشتن در حافظه



در بخش زیر به تشریح بلوک دیاگرام های پردازنده طراحی شده می پردازیم:

- ماژول imem :

تعریف ماژول در وریلاگ به صورت زیر می باشد:

```
module async_imem
(
    input [31:0] address,
    output [127:0] read_data
);
```

این ماژول قابلیت خواندن ۴ کلمه اطلاعات مجاور هم را دارد. با دادن آدرس به ورودی این ماژول اطلاعات خروجی قابل مشاهده است.

- ماژول FIFO Buffer :

تعریف این ماژول در وریلاگ به صورت زیر است:

```
module fifo_buffer
(
    input clk,
    input reset,
    input [31:0] In0,
    input [31:0] In1,
    input [31:0] In2,
    input [31:0] In3,
    input [2:0]RD,
    input [2:0]WR,
    output [31:0] Out0,
    output [31:0] Out1,
    output [31:0] Out2,
    output [31:0] Out3,
    output [3:0] capacity,
    output empty,
    output full
);
```

وظیفه این ماژول این است که اطلاعات ورودی از سمت imem را در یک بافر ۴ کلمه ای ذخیره کند. این بافر به صورت FIFO عمل میکند تا بتوانیم دستورات رو به ترتیبی که وارد شده است از آن خارج کنیم. همچنین این ماژول یک ورودی RD دارد که مشخص میکند هنگام کلاک خوردن چند کلمه از اطلاعات آن خوانده و از حافظه پاک شود و همچنین دارای یک ورودی WR می باشد که مشخص میکند در هنگام خوردن کلاک چند کلمه اطلاعات قرار است در آن نوشته شود. این ماژول دارای خروجی های Capacity, empty, full می باشد که مشخص میکند آیا حافظه خالی است یا پ و این که چه مقدار ظرفیت خالی دارد.

- ماژول FIDS :

تعریف این ماژول در زبان ورپلاگ به صورت زیر است:

```
module FourIssueDataSelector
(
input [31:0]D0,
input [31:0]D1,
input [31:0]D2,
input [31:0]D3,
input D1_Brnach_Mem,
input D2_Brnach_Mem,
input D3_Brnach_Mem,
input [2:0]IO_SEL,
input [2:0]I1_SEL,
input [2:0]I2_SEL,
input [2:0]I3_SEL,
output reg[31:0]IO,
output reg[31:0]I1,
output reg[31:0]I2,
output reg[31:0]I3,
output reg I1_Branch_Act,
output reg I2_Branch_Act,
output reg I3_Branch_Act
);
```

وظیفه این ماژول این است تا ۴ دستور ورودی که به آن وارد می شود را مسیر دهی کرده تا در استیج بعد هر دستور بسته به نوع آن وارد یکی از مسیر های Integer Branch, Integer, LW, SW بشود. این ماژول در واقع یک دی مالتی پلکسر است و سیگنال کنترلی آن که جهت مسیر اطلاعات ورودی را مشخص میکند توسط ماژول Program controller به این ماژول داده میشود تا اطلاعات در مسیر مشخص به خروجی بروند. همچنین در این ماژول اطلاعات مربوط به Branch Action بودن دستورات ورودی ۲ تا ۴ نیز جهت دهی می شوند. این سیگنال مشخص میکند که آیا دستور مورد نظر بعد از دستور Branch قرار گرفته است یا خیر. اگر دستوری بعد از دستور Branch قرار گرفته باشد باید این سیگنال ۱ منطقی شود تا زمانی که Branch taken رخ داد دستورات بعد از آن که در استیج Execution در حال اجرا هستند flush بشوند.

- ماژول Program Controller :

تعریف این ماژول در ورپلاگ به صورت زیر است :

```
module Issue_Controller
(
input [31:0]I1,
input [31:0]I2,
input [31:0]I3,
input [31:0]I4,
input fifo_empty,
output reg [2:0]fifo_ReadLine,
output [2:0]DataSel_1,
output [2:0]DataSel_2,
output [2:0]DataSel_3,
output [2:0]DataSel_4,
output reg I2_Branch_Mem,
output reg I3_Branch_Mem,
```

```
output reg l4_Branch_Mem,  
output reg [3:0]BranchPlusX  
);
```

این ماژول بخش اصلی پردازنده 4 issue را تشکیل میدهد. وظیفه این ماژول این است که تعیین کند چه دستوراتی می توانند با یکدیگر اجرا شوند و سپس دستوراتی که قابلیت اجرا شدن با یکدیگر را دارند از FIFO Buffer به استیج Execution هدایت کند. ورودی این ماژول 4 دستور از خروجی FIFO Buffer می باشد که پس از بررسی نوع دستورات و وابستگی های آن ها، مشخص می شود که اولاً چند دستور از 4 دستور ورودی باید به استیج بعدی بروند و ثانیاً مسیر دهی هر کدام از این دستورات به مسیر های Integer Branch, Integr, LW, SW را مشخص میکند. همچنین این ماژول تعداد دستوری که باید از FIFO Buffer خوانده شوند تا به استیج بعدی بروند را در پورت fifo_ReadLine مشخص میکند که این پورت به پورت RD بافر FIFO متصل میشود. همچنین این ماژول وضعیت Branch_Act بودن دستورات 2 تا 4 را نیز مشخص میکند که اگر هر کدام از دستورات 2 تا 4 بعد از دستور Branch آمده باشند این سیگنال ها 1 منطقی شده تا در استیج بعد اگر Branch taken رخ داد، دستوراتی که بعد از برنج آمده بودند را flush کنیم. همچنین این ماژول دارای یک خروجی BranchPlusX نیز میباشد. این عبارت یک ضریب تصحیح برای ساخت آدرس Branch میباشد که از آن استفاده میکنیم. همانطور که میدانیم ما همواره با یک آدرس PC، یک بلاک 4 کلمه ای از کلمات مجاور این آدرس را میخوانیم و به FIFO Buffer تحویل میدهیم. حال اگر دستور Branch از این بلاک 4 کلمه ای در جایگاهی غیر از کلمه 1 بود باید بدانیم که آدرس PC مربوط به این دستور به اندازه مضارب 4 بیشتر است. پس این را باید توسط Program controller بفهمیم و این ضریب تصحیح را تولید کنیم.

• ماژول Mem to Buffer Data Selector

این ماژول وظیفه مسیر دهی داده های خروجی imem به سمت FIFO Buffer را دارد. همانطور که میدانیم در حالت معمول 4 دستوری که از imem خوانده میشوند باید به همان ترتیب به 4 ورودی FIFO Buffer داده شوند. همانطور که میدانیم گام پرش آدرس PC برای imem به صورت 4 کلمه 4 کلمه می باشد. حال اگر به دلیل رخ دادن Branch ما به آدرسی غیر از مضارب 4 کلمه پرش کنیم باید بدانیم که ممکن است دستوراتی که از imem خارج میشوند از گام صحیح شروع شوند. به همین دلیل ما باید توسط یک ماژول و با بررسی آدرس PC بفهمیم که دقیقاً دستور شروع ما در خروجی imem کدام است و به طور مثال اگر 4 دستور در خروجی imem داریم، صفا دستور 2 به بعد را به Buffer تحویل دهیم. کار ماژول ذکر شده این است.

• ماژول PC Branch Generator

وظیفه این ماژول دو چیز است. یک تولید آدرس پرش و دوم این که اگر Branch taken رخ داد مراتب لازم برای اعمال آدرس مورد نظر به PC و imem و همچنین flush کردن دستورات رخ دهد. این ماژول با دریافت وضعیت Branch_e و یا Branch_n از controller و همچنین وضعیت Zero بودن خروجی alu اقدام به تولید سیگنال Branch taken میکند و در همین حین آدرس پرش را با توجه به مقدار PC_D و BranchPlusX_D میسازد.

با توجه به این که فرایندهای Decode و Execution و WriteBack در یک استیج اجرا می شوند، دیگر بحث Hazard مطرح نیست و نیازی به data forwarding و Stalling در این استیج وجود ندارد.

نکته قابل توجه در این استیج این است که چون 3 پورت نوشتن همزمان وجود دارد بایستی حتماً به وابستگی Write after write توجه شود و اگر دو یا سه دستور Write مربوط به یک آدرس وجود داشت حتماً توالی دستورات از قبل بررسی شده باشد تا آخرین دستور همواره مقدار خود را در حافظه بنویسد.

وضعیت خروجی تست بنچ بعد از شبیه سازی به صورت زیر می باشد:

```
#  
# PASS1, All Sorted!  
#  
# Pass2, Output Matches the expected Numbers!  
#  
#  
# ** Note: $stop      : D:/University/Courses/Advanced Computer Architecture/Projects/03_FourIssue_Imp/MIPS_FourIssue@tb.v(182)  
#    Time: 260250 ns  Iteration: 3  Instance: /MIPS_FourIssue_tb
```

مدت زمان اجرای برنامه : 260250ns